Unit – 3
Error Handling

Prepared By: Prof. Meghna K. Bhatt

fppt.com

Course content

- Exceptions
- Utility of Exceptions
- Built-in Exceptions
- User-defined Exceptions
- User-defined Exceptions, Pragma EXCEPTION_INIT,
- RAISE_APPLICATION_ERROR procedure
- SQLCODE and SQLERRM

fppt.com

What is exceptions?

- Exception means an error.
- An exception is an error condition during a program execution.
- Exceptions are runtime errors or unexpected events that occur during the execution of a PL/SQL code block.
- An exception is an error which disrupts the normal flow of program instructions.

fppt.com

What is exception?

- An exception occurs when the PL/SQL engine encounters an instruction which it cannot execute due to an error that occurs at run-time.
- These errors will not be captured at the time of compilation and hence these needed to handle only at the run-time.
- For example:
  - PL/SQL engine receives an instruction to divide any number by ‘0’, then the PL/SQL engine will throw it as an exception.
  - The exception is only raised at runtime during the execution.

fppt.com

What is Exception Handling?

- Exceptions will stop the program from executing further, so to avoid such condition, they need to be captured and handled separately.
- This process is called as Exception-Handling, in which the programmer handles the exception that can occur at the run time.

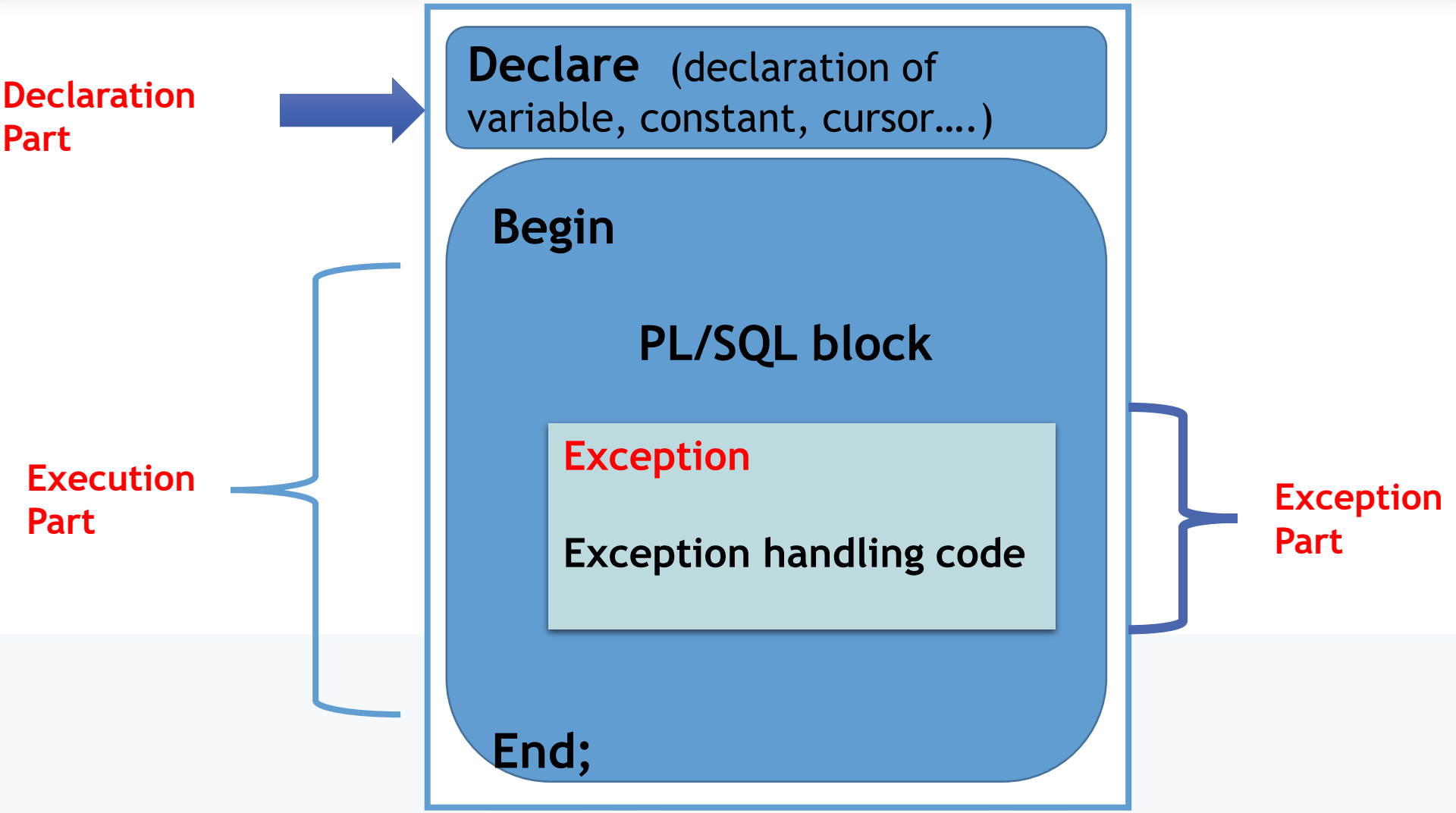
fppt.com

What is Exception Handling?

- When exceptions occurs :
  - The oracle engine is the first one to identify such an exception
  - it immediately tries to resolve it by default exception handler.
- PL/SQL supports programmers to catch such conditions using **EXCEPTION** block which is written in **Execution** Block of PL/SQL program & an appropriate action is taken against the error condition.

fppt.com

Exception Handling Block



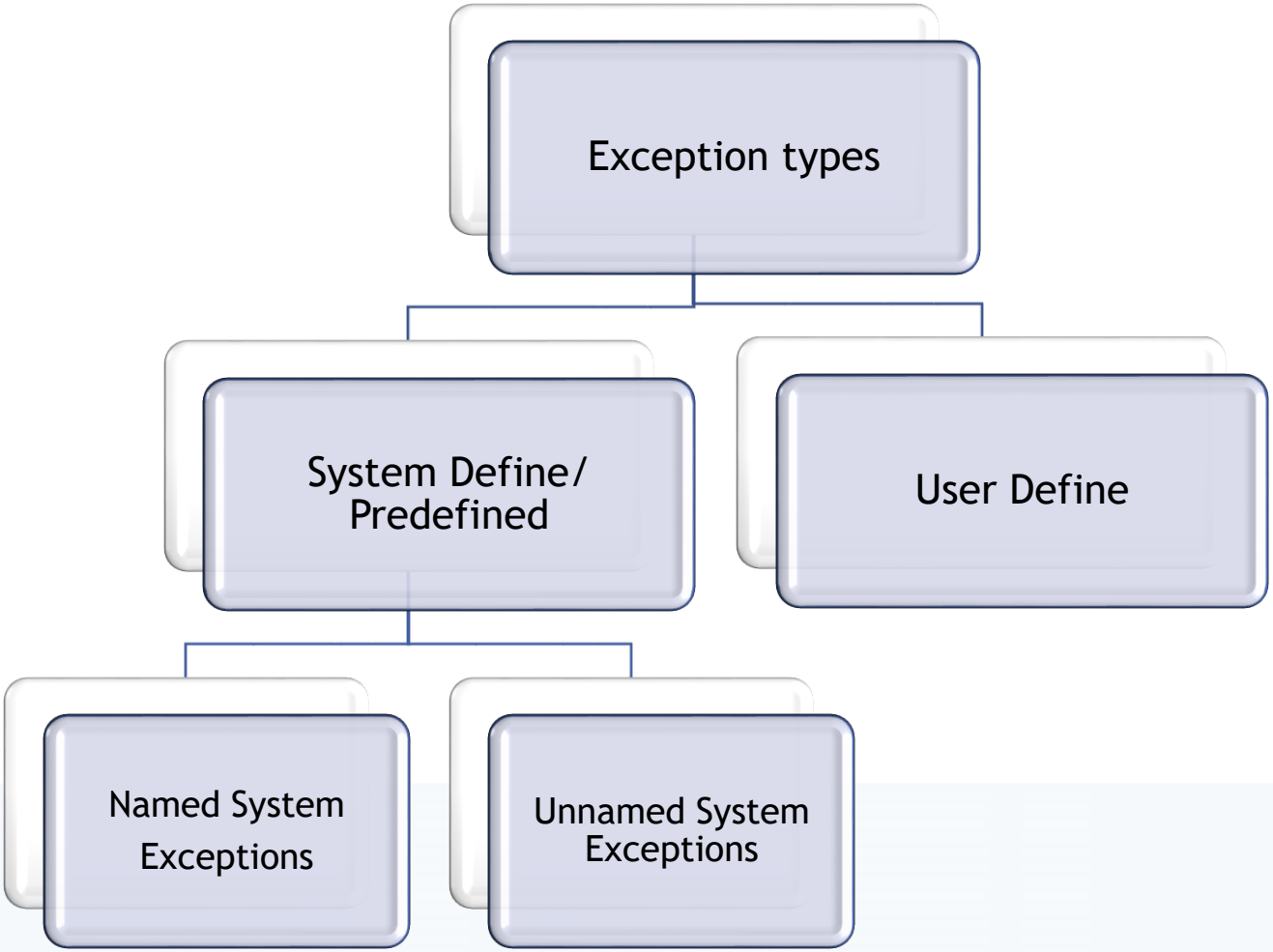
fppt.com

Exception Handling Block

```
DECLARE
    <declarations section>
BEGIN
    <executable command(s)>
EXCEPTION
    <exception handling goes here >
    WHEN exception1 THEN
        exception1-handling-statements
    WHEN exception2  THEN
        exception2-handling-statements
    WHEN exception3 THEN
        exception3-handling-statements
    ........
    WHEN others THEN
        exception3-handling-statements
END;
```

fppt.com

Types of Exceptions



fppt.com

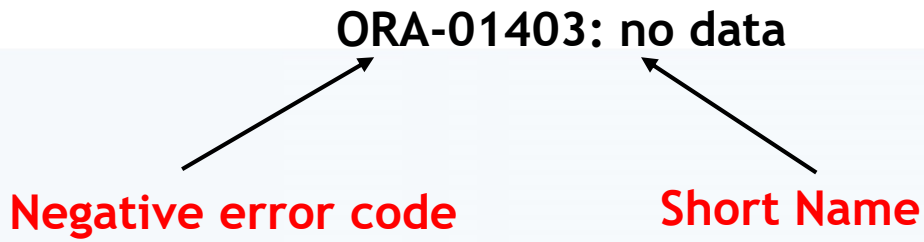
System Defined Exception

- These exceptions are predefined in PL/SQL which get raised WHEN certain **database rule is violated.**
- The system-defined exceptions are defined and maintained implicitly by the oracle server.
- They are defined in oracle standard package.
- Whenever an exception occurs inside the program the oracle server matches and identifies the appropriate exception from the available set of exceptions.

fppt.com

Named System Exception

- Exceptions that have been given names by PL/SQL and raised as a result of an error in PL/SQL.
- These exceptions have a negative error code associated with it.
- In addition to the error code and error message, the system-defined exceptions have a short name which is used with the exception handlers.
- Its defined in oracle **standard** package.
- Example :



fppt.com

Named System Exception

Exception	Oracle Error	Description
DUP_VAL_ON_INDEX	00001	It is raised when duplicate values are attempted to be stored in a column with a unique index.
INVALID_CURSOR	01001	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
CURSOR_ALREADY_OPEN	06511	This exception is raised when user tries to open a cursor that is already open
INVALID_NUMBER	01722	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.
LOGIN_DENIED	01017	It is raised when a program attempts to log on to the database with an invalid username or password.

fppt.com

Named System Exception

NO_DATA_FOUND	01403	It is raised when a SELECT INTO statement returns no rows.
NOT_LOGGED_ON	01012	It is raised when a database call is issued without being connected to the database.
PROGRAM_ERROR	06501	It is raised when PL/SQL has an internal problem.
ROWTYPE_MISMATCH	06504	It is raised when a cursor fetches value in a variable having incompatible data type.

fppt.com

Named System Exception

STORAGE_ERROR	06500	It is raised when PL/SQL ran out of memory or memory was corrupted.
TOO_MANY_ROWS	01422	It is raised when a SELECT INTO statement returns more than one row.
VALUE_ERROR	06502	It is raised when an arithmetic, conversion, truncation, or size constraint error occurs.
ZERO_DIVIDE	01476	It is raised when an attempt is made to divide a number by zero.
OTHERS	--	This flag is used to catch any error situation not coded by the programmer in the exception section. Therefore, it must appear last in the exception section.

fppt.com

Unnamed System Exceptions

- The system exceptions for which ***Oracle does not have a name*** are known as unnamed system exceptions. ***These exceptions do not occur frequently*** and are written with a code and an associated message.

- Using the ***WHEN OTHERS*** exception handler we can handle the unnamed exceptions

fppt.com

Unnamed System Exceptions EXAMPLE

```
SET SERVEROUTPUT ON
BEGIN
        DBMS_OUTPUT.put_line(TO_DATE    ('2010    10    10
44:55:66', 'YYYSS'));
EXCEPTION
        WHEN others THEN
    DBMS_OUTPUT.put_line('Bad date format');
END;
```

fppt.com

USER DEFINE EXCEPTION

- Apart from system exceptions we can explicitly define ***exceptions based on business rules***. These are known as ***user-defined exceptions***.

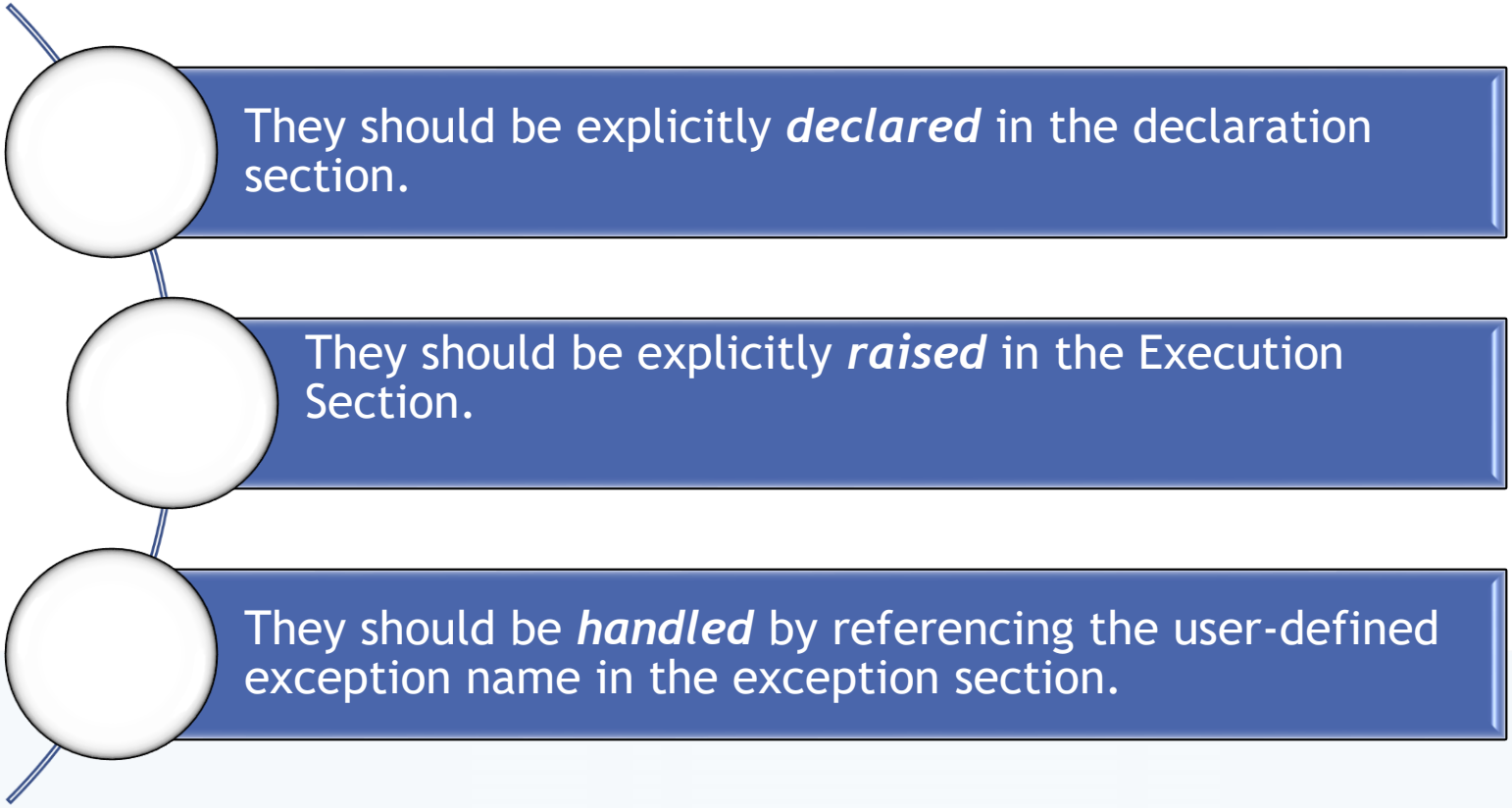
- Like all other programming languages, Oracle also allows you to ***declare and implement your own exceptions***.

- Unlike System defined exceptions, ***these exceptions are raised explicitly in the PL/SQL block***.

fppt.com

USER DEFINE EXCEPTION

- **Steps to be followed to use user-defined exceptions:**



fppt.com

SYNTAX:

```
Declare
        exeception_name EXCEPTION;
Begin
        if <Condition> then
                RAISE exception_name;
        end if;
        EXCEPTION
                WHEN exception_name THEN
                                Statement;
End;
```

fppt.com

Exception Trapping Functions

- When an exception occurs in your program, ***you can find the error code for the error and its associated message***. Once you know the error code and the message, you can modify your program to take action based on the error.
- The two functions to identify the error code and error message.
  - **SQLCODE:**
  - **SQLERRM:**

fppt.com

WHY SQLCODE & SQLERRM ARE IMPORNTANT?

- When a SQL statement raises an exception,
- Oracle captures the error codes by using the SQLCODE and SQLERRM globally-defined variables.
- SQLCODE and SQLERRM can track exceptions that are handled by the OTHERS clause of the exception handler.
- SQLCODE returns the current error code from the error stack and the error message from the current error.

fppt.com

SQLCODE

- SQLCODE function is used within the Exception Handling section of a code.
- It is used to get the error number.
- Here, the error number is associated with the most recently raised error exception.
- If the exceptions are internal, SQLCODE returns a negative number except for the NO_DATA_FOUND exception which has the number code +100.
- If the exception is user-defined, SQLCODE returns +1 or the number that you associated with the exception via the pragma EXCEPTION_INIT.

fppt.com

SQLCODE EXAMPLE

```
DECLARE l_code NUMBER;
        r_customer customers%rowtype;
BEGIN
        SELECT * INTO r_customer FROM customers;
EXCEPTION
WHEN OTHERS THEN
        l_code := SQLCODE;
dbms_output.put_line('Error code:' || l_code);
END;
/
```

fppt.com

SQLERRM

- The function SQLERRM takes an argument as an error number and returns the error message associated with that error number

- SYNTAX:

SQLERRM([error_number])

- If you omit the error_number argument, the function will return the error message associated with the current value of SQLCODE.

fppt.com

Example

```
set serveroutput on
declare
        a number:=&a;
        b number:=&b;
        c number;
begin
   c:=a/b;
exception
        when ZERO_DIVIDE THEN
                dbms_output.put_line('cant divide no with
o'||sqlerrm);
End;
/
```

fppt.com

Raise_Application_Error ( ) Procedure

- ***RAISE_APPLICATION_ERROR*** is a built-in procedure in oracle which is used to display the user-defined error messages along with the error number whose ***range is in between -20000 and -20999.***

- Whenever a message is displayed using ***RAISE_APPLICATION_ERROR***, all previous transactions ***which are not committed within the PL/SQL Block are rolled back automatically*** (i.e. change due to INSERT, UPDATE, or DELETE statements).

- ***RAISE_APPLICATION_ERROR*** raises an exception but does not handle it.

- ***RAISE_APPLICATION_ERROR*** is used for the following reasons,
  a) to create a unique id for an user-defined exception.
  b) to make the user-defined exception look like an Oracle error.

fppt.com

Raise_Application_Error ( ) Procedure

- **The General Syntax to use this procedure is:**
  - ***Syntax:***

    RAISE_APPLICATION_ERROR (error_number, error_message);

  - The error_number must be between **-20000 and -20999**
  - The error_message is the message you want to display when the error occurs.
  - The message is a character string that represents the error message. Its length is up to 2048 bytes.

fppt.com

Raise_Application_Error ( ) Procedure

- **Steps to be followed to use RAISE_APPLICATION_ERROR procedure:**

1. Declare a user-defined exception in the declaration section
2. Raise the user-defined exception based on a specific business rule in the execution section.
3. catch the exception and link the exception to a user-defined error number in RAISE_APPLICATION_ERROR

fppt.com

Raise_Application_Error ( ) Procedure

Example 9:

```
--Write a program that explains the use of ZERO_DIVIDE exception.
SET SERVEROUTPUT ON
DECLARE
    N     NUMBER (3);
    VD    NUMBER(3);
BEGIN
    VD:=&VD;
    IF VD=0 THEN
          RAISE_APPLICATION_ERROR(-20230,'You can not divide any number
    with zero');
    ELSE
          SELECT 100/VD INTO N FROM DUAL;
          DBMS_OUTPUT.PUT_LINE(N);
    END IF;
END;
/
```

fppt.com

Raise_Application_Error ( ) Procedure

Example 10:

```
--Write a program that explains the use of ZERO_DIVIDE exception.
SET SERVEROUTPUT ON
DECLARE
    N     NUMBER (3);
    VD    NUMBER(3);
BEGIN
    VD:=&VD;
    SELECT 100/VD INTO N FROM DUAL;
    DBMS_OUTPUT.PUT_LINE(N);

EXCEPTION
    WHEN OTHERS THEN
          RAISE_APPLICATION_ERROR(-20230,'You can not divide any number
    with zero');
END;
/
```

fppt.com

Raise_Application_Error ( ) Procedure

Example 11:

--Write a program that explains the use of ZERO_DIVIDE exception.

```
SET SERVEROUTPUT ON
DECLARE
    N       NUMBER (3);
    VD      NUMBER(3);
    E1      EXCEPTION;
BEGIN
    VD:=&VD;
    IF VD=0 THEN
            RAISE E1;
    ELSE
            SELECT 100/VD INTO N FROM DUAL;
            DBMS_OUTPUT.PUT_LINE(N);
    END IF;
EXCEPTION
    WHEN E1 THEN
            RAISE_APPLICATION_ERROR(-20230,'You can not divide any number with zero');
END;
/
```

fppt.com

Pragma EXCEPTION_INIT

- PL/SQL user named exception.
- you can define your own error message and error number using Pragma EXCEPTION_INIT or RAISE_APPLICATION_ERROR function.
- Pragma is a keyword directive to execute proceed at compile time.

fppt.com

Pragma EXCEPTION_INIT

- pragma EXCEPTION_INIT function take this two arguments:
    - exception_name
    - error_number
- define pragrma EXCEPTION_INIT in DECLARE BLOCK on your program.
- Syntax:

```
PRAGMA EXCEPTION_INIT(exception_name, -error_number);
```

fppt.com

Example

```
set serveroutput on
declare
            a number:=&a;
            b number:=&b;
            c number;
            myexp1 EXCEPTION;
            PRAGMA
EXCEPTION_INIT(myexp1, -2000);
 begin
   if b=0 then
         RAISE myexp1;
else
          c:=a/b;
          dbms_output.put_line('ans='||c);
end if;
exception
when myexp1 THEN
                dbms_output.put_line('cant
divide no with o');
end;
/
```

fppt.com

Example

```
DECLARE
            myex EXCEPTION;
           PRAGMA EXCEPTION_INIT(myex,-20015);
                    n NUMBER := &n;
BEGIN
   FOR i IN 1..n
LOOP
            dbms_output.put.line(i);
    IF i=n THEN
          RAISE myex;
       END IF;
   END LOOP;
EXCEPTION
   WHEN myex THEN
       dbms_output.put.line('loop finish');
END;
/
```

fppt.com